

TECNOLÓGICO NACIONAL DE MÉXICO

Gestión de Proyectos

DISEÑO DE ARQUITECTURA DE SEGURIDAD (MIDDLEWARE)

Equipo 3A

Bolaños García Omar Gadiel

Diego Emilio Ortiz Vilchez

Perez Manriques Jorge Luis

Chavez Vega Emilio Sebastian

ING. Sistemas Computacionales | Celaya

Proyecto: Monitor de Nivel de Agua en Tinaco

KR 3.4: Diseño de Arquitectura de Seguridad (Middleware)

Entregable: Diagrama de flujo de autenticación/validación

Sprint 3 | Fecha: 13 may 2026 | **Estado:** In Review

1. Objetivo

Implementar una capa de seguridad (Middleware) para el endpoint de inserción de datos, garantizando que solo el hardware autorizado (ESP32) pueda modificar el estado del sistema. Se busca prevenir ataques de inyección de datos falsos y asegurar la integridad de la información almacenada en Firebase Realtime Database.

2. Mecanismos de Seguridad Implementados

- Firestore Auth Token (Database Secret):** El ESP32 debe incluir una clave secreta en cada petición PUT. Sin esta clave, las Reglas de Seguridad de Firestore rechazarán la conexión.
- Validación de Esquema:** El sistema verificará que el JSON contenga exactamente las claves definidas en el KR 3.2 (d, n, p, a, t, s).
- Filtrado de Rangos:** Se descartan lecturas físicamente imposibles (ej. porcentajes negativos o superiores a 100).

3. Diagrama de Flujo de Autenticación y Validación

El siguiente flujo describe el procesamiento de una petición desde el hardware hasta la base de datos:

Inicio de Transmisión (ESP32)



4. Fragmento de Lógica de Seguridad (Pseudo-código Middleware)

Este fragmento ilustra la lógica que debe ejecutarse antes de aceptar los datos en el backend o mediante reglas de base de datos:

```
// Reglas de Seguridad de Firebase (JSON)
{
  "rules": {
    "tinaco": {
      "lectura": {
        ".read": "auth != null",
        ".write": "auth != null &&
                  newData.hasChildren(['d', 'n', 'p', 'a', 't', 's'])",
        "p": {
          ".validate": "newData.isNumber() &&
                      newData.val() >= 0 &&
                      newData.val() <= 100"
        }
      }
    }
  }
}
```

5. Conclusiones

La arquitectura propuesta establece un perímetro de seguridad robusto sin añadir latencia significativa. La validación delegada en las **Firebase Security Rules** permite que el filtrado ocurra en la capa de red de Google, liberando recursos de procesamiento en el microcontrolador y garantizando que solo datos confiables lleguen al usuario final.